Security Audit Report

f(x) Protocol 2.0 For Katana Chain

by AladdinDAO

SECBIT

March 9, 2026

1. Introduction

AladdinDAO is a decentralized network that seeks to shift crypto investment opportunities from venture capitalists to the wisdom of crowds through collective value discovery. Building on the foundation of version 1.0, f(x) Protocol 2.0 retains its stablecoin component (`fxUSD`) while upgrading its leveraged position mechanism (`xPOSITION`). The protocol has already been deployed and operated on Ethereum. For Katana, the design is streamlined into a lightweight, mint-focused deployment with no native trading, no looping, and limited reliance on external DeFi infrastructure. To support this deployment, the team also introduced Katana-specific oracle components and related infrastructure adjustments. SECBIT Labs conducted this audit from February 27, 2026 to March 9, 2026, covering three areas: **code bugs**, **logic flaws**, and **risk assessment**. The assessment indicates that f(x) Protocol 2.0 for the Katana chain has no critical security risks. The SECBIT team also identified several implementation and design considerations for improvement (see Section 4 for details).

Type	Description	Level	Status
Design & Implementation	4.3.1 During protocol initialization, fxUSD tokens are minted without collateral to the specified `_recipient` address, which violates the protocol's fundamental requirement of full collateralization.	Info	Discussed
Design & Implementation	4.3.2 Remove unnecessary fToken logic for Katana chain deployment.	Info	Discussed
Design & Implementation	4.3.3 Remove the `checkpoint()` function to save gas.	Info	Discussed
Design & Implementation	4.3.4 Remove the initial values declared for parameters such as `fxUSD` to avoid misunderstandings.	Info	Fixed

2. Contract Information

This part describes the basic contract information and code structure.

2.1 Basic Information

The basic information about the f(x) Protocol 2.0 for the Katana chain is shown below:

- Smart contract code
  - initial review commit *1a4591d*
  - final review commit *c5c9bf2*

2.2 Contract List

The following table lists the contracts within the review scope for f(x) Protocol 2.0 on the Katana chain:

Name	Lines	Description
MorphoFundingPool.sol	60	This funding pool updates the collateral index and applies funding costs based on the configured interest rate.
FxUSDPriceOracle.sol	119	This contract provides the `fxUSD/USD` price feed on the Katana chain using Sushi TWAP and Chainlink `USDC/USD`.
FxUSDRegeneracy.sol	323	This contract facilitates the minting of fxUSD on the Katana chain. Note that all fToken-related features are non-functional and have not been enabled in this implementation.
WeETHRateProvider.sol	12	This helper contract provides the `weETH/ETH` rate used by the Katana `weETH` pool for token scaling.
PegKeeper.sol	119	This auxiliary module supports peg maintenance between `fxUSD` and `USDC`.
PoolConfiguration.sol	263	This contract manages fee parameters, peg-related gates, and the manually updated interest-rate snapshot.
PoolManager.sol	728	This core contract manages position operations, redemption, rebalancing, liquidation, and harvest-related accounting.
MultiPathConverter.sol	63	This generalized swap helper supports configured asset-conversion routes used by peripheral and peg-keeping flows.
FxUSDBasePoolFacet.sol	65	This helper contract converts supported assets into `fxUSD` or `USDC` for deposit into the `FxUSDBasePool`.
PositionOperateFacet.sol	133	This router facet helps users borrow and repay positions with supported input or output assets.

Notice: The scope of this engagement is limited to the incremental updates introduced for the Katana deployment of f(x) Protocol 2.0. The specific code changes under review are enumerated above.

3. Contract Analysis

This part describes code assessment details, including "role classification" and "functional analysis".

3.1 Role Classification

Two key roles in the f(x) Protocol 2.0 for the Katana chain are Governance Account and Common Account.

- Governance Account
  - Description

    Protocol administrator
  - Authority
    - Update fee ratios
    - Update market and incentive configurations
    - Transfer ownership
    - Pause crucial functions
  - Method of Authorization

    The governance account is initially assigned to the deployer and may be transferred by the current administrator.
- Common Account
  - Description

    Users of f(x) Protocol 2.0 on the Katana chain
  - Authority
    - Open and close positions with supported tokens
    - Redeem collateral with `fxUSD`
    - Rebalance user positions to prevent excessive debt ratios
    - Liquidate positions and participate in protocol bad-debt resolution
  - Method of Authorization

    No authorization required

3.2 Functional Analysis

The f(x) protocol implements a decentralized quasi-stablecoin with efficient collateral utilization and leveraged-position infrastructure. The Katana deployment preserves the core `fxUSD` minting and position-management logic while streamlining the overall architecture for a mint-focused environment. It removes native trading and looping features while keeping the oracle, funding, redemption, rebalance, and liquidation paths necessary for operation. The SECBIT team conducted a detailed audit of the contracts within this review scope.

We can divide the critical functions relevant to this review into five parts:

PoolManager

The contract provides the core interfaces for position operation, redemption, rebalancing, liquidation, and funding-related accounting. The main functions in this contract are as follows:

- `operate()`

  The function enables users to initiate positions and execute position adjustments.

- `redeem()`

  Users holding `fxUSD` can invoke this function to redeem collateral assets when redemption is enabled.

- `rebalance()`

  This function is used to reduce unhealthy position debt when collateral-price movements cause debt ratios to rise beyond the rebalance threshold.

- `liquidate()`

  This function is used through the `fxBASE` path to liquidate positions whose debt ratios exceed the liquidation threshold.

- `harvest()`

  An authorized harvester can invoke this function to settle funding and reward accounting and receive the configured harvester incentive.

FxUSDBasePool

The contract implements the base-pool accounting used for `fxUSD`/`USDC` liquidity management, redemption, rebalancing, liquidation, and peg support. The main functions in this contract are as follows:

- `deposit()`

  Through this function, any user can deposit fxUSD tokens or USDC tokens and receive proportional fxUSD base pool token shares in return.

- `redeem()`

  Users can redeem their shares through this function based on the current ratio of fxUSD tokens and USDC tokens within the contract.

- `rebalance()`

  When protocol position debt ratios reach the rebalance threshold, users can invoke this function to redeem position debt and reduce debt ratios, primarily utilizing fxUSD tokens held within the contract.

- `liquidate()`

  When protocol position debt ratios reach the liquidation threshold, users can invoke this function to execute position liquidation, primarily utilizing fxUSD

tokens held within the contract.

- `arbitrage()`

  The `PegKeeper` can invoke this function to perform stabilization trades between `fxUSD` and `USDC`.

FxUSDBasePoolFacet

This helper contract for `FxUSDBasePool` facilitates the conversion of supported external assets into `fxUSD` or `USDC` for subsequent deposit into `FxUSDBasePool`. The main functions in this contract are as follows:

- `migrateToFxBase()`

  Users holding legacy assets can utilize this function to withdraw and convert their holdings into fxUSD tokens, which they can then deposit into the `FxUSDBasePool` contract.

- `migrateToFxBaseGauge()`

  Users holding legacy assets can utilize this function to withdraw and convert their holdings into fxUSD tokens for deposit into the `FxUSDBasePool` contract. Subsequently, the received fxUSD base pool token shares are deposited into corresponding gauges to earn FXN token rewards.

- `depositToFxBase()`

  Users can convert their external assets into fxUSD tokens for subsequent deposit into the fxUSD base pool.

- `depositToFxBaseGauge()`

  Users can convert their external assets into fxUSD tokens, which they deposit into the fxUSD base pool. Subsequently, the received shares are deposited into corresponding gauges to earn FXN token rewards.

- `redeemFromFxBase()`

  Users can burn their fx base pool shares to retrieve corresponding fxUSD tokens and USDC tokens.

- `redeemFromFxBaseGauge()`

  Users can withdraw fx base pool shares from gauges and burn them to retrieve fxUSD tokens and USDC tokens from the `FxUSDBasePool` contract.

FxUSDPriceOracle

Because Katana does not provide a native on-chain oracle for `fxUSD`, the protocol relies on a Sushi pool for price discovery and uses its TWAP, together with Chainlink `USDC/USD`, to derive the `fxUSD/USD` price. The main function in this contract is as follows:

- `getPrice()`

  This function returns the derived `fxUSD` price together with a boolean flag indicating whether the token remains within the configured peg range.

WeETHRateProvider

This helper contract provides the `weETH/ETH` rate used by the Katana `weETH` pool for collateral scaling. The main function in this contract is as follows:

- `getRate()`

  This function returns the Chainlink-based `weETH/ETH` rate used by the protocol.

4. Audit Detail

This section describes the audit process and presents the detailed findings and potential risks.

4.1 Audit Process

The audit strictly followed the audit specification of SECBIT Labs. We analyzed the project from the perspectives of code bugs, logical implementation, and potential risks. The process consisted of four steps:

- Full line-by-line analysis of the contract code.
- Evaluation of vulnerabilities and potential risks revealed by the contract code.
- Communication of findings and confirmation with the development team.
- Audit report preparation.

4.2 Audit Result

After scanning with adelaide, sf-checker, and badmsg.sender (internal version) developed by SECBIT Labs, together with open-source tools including Mythril, Slither, SmartCheck, and Securify, the auditing team performed a manual assessment. The team inspected the contracts line by line, and the results can be categorized as follows:

Number	Classification	Result

1	Features defined by the contracts function normally	✓
2	No obvious bug (e.g., overflow, underflow)	✓
3	Passes Solidity compiler checks with no apparent error	✓
4	Passes common tool checks with no obvious vulnerability	✓
5	No obvious gas-consuming operation	✓
6	ERC20-related implementations comply with the standard	✓
7	No risk in low-level call (call, delegatecall, callcode) and in-line assembly	✓
8	No deprecated or outdated usage	✓
9	Explicit implementation, visibility, variable type, and Solidity version number	✓
10	No redundant code	✓
11	No potential risk manipulated by timestamp and network environment	✓
12	Explicit business logic	✓
13	Implementation consistent with annotation and other info	✓
14	No hidden code about any logic that is not mentioned in the design	✓
15	No ambiguous logic	✓
16	No risk threatening the developing team	✓
17	No risk threatening exchanges, wallets, and DApps	✓
18	No risk threatening token holders	✓
19	No privilege on managing others' balances	✓
20	No hidden minting path beyond the documented initialization flow	✓
21	Correct privilege-management hierarchy	✓

4.3 Issues

4.3.1 During protocol initialization, fxUSD tokens are minted without collateral to the specified `_recipient` address, which violates the protocol's fundamental requirement of full collateralization.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Info	Design logic	Discussed

Location

FxUSDRegeneracy.sol#L169

Description

Upon initialization, this contract mints a specified amount of `fxUSD` to a designated `_recipient` address. Based on the deployment parameters and logs in the reviewed repository, this initial supply is explicitly configured. Even if the deployment is intended only for testing or bootstrap purposes, it means the default workflow includes an uncollateralized initial supply. Because these `fxUSD` tokens are minted without corresponding collateral, they constitute an exogenous initial supply within the system. They could later enter `fxBASE` and dilute genuine `USDC` liquidity deposited by future users, or be used through `PoolManager.redeem()` to extract collateral supplied by later borrowers once redemption conditions are met. This conflicts with the protocol's stated full-collateralization model.

```
function initialize(
    string memory _name,
    string memory _symbol,
    uint256 _initSupply,
    address _recipient
  ) external initializer {
    __Context_init();
    __ERC165_init();
    __AccessControl_init();
    __ERC20_init(_name, _symbol);
    __ERC20Permit_init(_name);

    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());

    // old init v2
    stableReserve.decimals =
FxUSDRegeneracy(stableToken).decimals();
```

```
    // @audit as fTokens do not exist on the Katana chain, this
value remains consistently at zero. Consequently, all logic
associated with fTokens is effectively dormant and will not be
executed, which aligns with the intended design.
    legacyTotalSupply = totalSupply();

    // init supply
    // @audit this flaw allows for the generation of
uncollateralized fxUSD, which can be utilized as initial capital
to drain genuine USDC or other collateral from the protocol.
    _mint(_recipient, _initSupply);
  }
```

Status

The developers confirmed the finding and explained that the pre-minted supply is intended to bootstrap the `fxUSD/USDC` trading pair. In the early stage of deployment, redemption based on secondary-market purchases will remain disabled to protect collateral integrity, while primary functions such as opening and closing positions will remain available. In the final commit c5c9bf2, the developers also added a `MINTER_ROLE`-gated `burn(uint256 amount)` function to support retirement of the seeded `fxUSD`. The team stated that it will keep the initial uncollateralized amount sufficiently small and, once market liquidity becomes adequate, recover and burn the seeded amount before enabling redemption.

4.3.2 Remove unnecessary fToken logic for Katana chain deployment.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Info	Design logic	Discussed

Description

Many core functions in `FxUSDRegeneracy` are designed for fToken-related flows, which are not supported in the current Katana deployment. To reduce attack surface and improve deployment efficiency, the development team should consider removing or disabling these unnecessary code paths. Notably, the `wrapFrom()` function had an issue that has already been fixed in the main branch. However, because the prerequisite fToken infrastructure is not enabled on Katana, the conditions required for that issue to arise do not currently exist.

```
function wrapFrom(address _pool, uint256 _amount, address
_receiver) external override onlySupportedPool(_pool) {
    if (isUnderCollateral()) revert ErrorUnderCollateral();

    address _baseToken =
IFxShareableRebalancePool(_pool).baseToken();
    _checkBaseToken(_baseToken);
    _checkMarketMintable(_baseToken, false);

    IFxShareableRebalancePool(_pool).withdrawFrom(_msgSender(),
_amount, address(this));
    _mintShares(_baseToken, _receiver, _amount);

    emit Wrap(_baseToken, _msgSender(), _receiver, _amount);
}
```

Status

The development team confirmed the observation, stating that the fToken-specific functions are redundant in the current Katana deployment and are not expected to be executed. The team decided to defer removal of this code until a later release.

4.3.3 Remove the `checkpoint()` function to save gas.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Info	Design logic	Discussed

Location

PoolConfiguration.sol#L333-L336

Description

On the Katana deployment, the funding rate is updated manually through `setInterestRate()`. As a result, the `checkpoint()` function no longer performs any meaningful rate-update logic in the current implementation and can be considered for removal to reduce technical debt and save gas.

```
/// @inheritdoc IPoolConfiguration
/// @dev It is ok to let anyone call this function. However to
reduce the unknown risk, we require the pool address is
/// the same as the caller and the pool is whitelisted.
function checkpoint(address pool) external {
  if (pool != _msgSender()) revert ErrorInvalidPool();
  if (poolFeeRatio[pool].defaultFeeRatio == bytes32(0)) revert
ErrorInvalidPool();
}
```

Status

The development team acknowledged this finding and decided to keep the code temporarily, with plans to address it more comprehensively in a future release.

4.3.4 Remove the initial values declared for parameters such as `fxUSD` to avoid misunderstandings.

Risk Type	Risk Level	Impact	Status
Design & Implementation	Info	Design logic	Fixed

Location

PositionOperateFacet.sol#L50-L59

Description

When the Solidity compiler version is 0.8.20 or higher, `immutable` parameters are handled by allocating temporary storage to collect their final values within the constructor. After the constructor finishes, these values are embedded directly into the runtime bytecode and the constructor code is discarded. As a result, the actual values of `fxUSD`, `longPoolManager`, `shortPoolManager`, and `configuration` depend solely on the constructor arguments rather than the initial values assigned at declaration. We recommend removing those redundant declaration-time initializations to improve code clarity and avoid misunderstanding.

```
/// @dev The address of fxUSD token.
address private immutable fxUSD =
0x085780639CC2cACd35E474e71f4d000e2405d8f6;

/// @dev The address of long pool manager.
address private immutable longPoolManager =
0x250893CA4Ba5d05626C785e8da758026928FCD24;

/// @dev The address of short pool manager.
```

```
address private immutable shortPoolManager =
0xaCDc0AB51178d0Ae8F70c1EAd7d3cF5421FDd66D;

/// @dev The address of PoolConfiguration.
address private immutable configuration =
0x16b334f2644cc00b85DB1A1efF0C2C395e00C28d;


// @audit the constructor explicitly initializes the fxUSD,
longPoolManager, shortPoolManager, and configuration addresses,
rendering the default values assigned during state variable
declaration obsolete
constructor(address _fxUSD, address _longPoolManager, address
_shortPoolManager, address _configuration) {
  fxUSD = _fxUSD;
  longPoolManager = _longPoolManager;
  shortPoolManager = _shortPoolManager;
  configuration = _configuration;
}
```

Status

The development team confirmed the issue and implemented a fix in commit b92a0d6.

5. Conclusion

After auditing and analyzing f(x) Protocol 2.0 for the Katana chain, SECBIT Labs identified several issues and optimization suggestions, as described above.

Disclaimer

SECBIT's smart contract audit service assesses contract correctness, security, and performance from the perspectives of code quality, logic design, and potential risks. This report is provided "as is", without any warranty regarding the code's practicality, the business model, the management system, or the suitability of the contracts for any particular purpose. This audit report must not be regarded as an endorsement of the platform, team, company, or any investment related to it.

APPENDIX

Vulnerability/Risk Level Classification

Level	Description
High	Severely damages contract integrity and may allow attackers to steal Ether or tokens, or lock assets inside the contract.
Medium	Damages contract security under certain conditions and may impair stakeholder interests.
Low	Causes no material impairment to the contract.
Info	Relates to best practices or design rationality and could potentially introduce risk.

SECBIT Labs is devoted to constructing a reliable and orderly blockchain economic entity based on common consensus.

https://secbit.io

audit@secbit.io

@secbit_io